

Table des matières

Exercices 01 — Paysage SQL et modèle relationnel	1
Questions courtes	1
Questions pratiques	3
Diagnostics	4
Questions à développer	5
Mises en situation	5

Exercices 01 — Paysage SQL et modèle relationnel

Comment travailler ces exercices. Répondez à chaque énoncé seul, puis vérifiez. Les questions courtes vérifient que le vocabulaire est en place ; les pratiques vous font écrire du SQL ; les diagnostics vous montrent du code qui cloche ; les mises en situation vous demandent de choisir une approche et de la défendre. Comptez quatre heures et quart pour y répondre, le corrigé non compris.

Les repères de terrain. Ce cours n'a qu'un laboratoire, sur l'entrepôt : les énoncés pratiques se traitent donc dans DuckDB, sur `adventureworksdw.duckdb` ouvert par `USE dbo;`, et portent la marque [OLAP]. Quelques questions parlent du monde transactionnel sans rien à exécuter : elles préparent le Cours 02, où le double chemin commence pour de bon.

Le corrigé est dans `Corrige_01.md`. Répondez à l'énoncé en entier avant de l'ouvrir : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Les cinq mots. Rangez data, database, DBMS, RDBMS et SQL du plus concret au plus abstrait, et donnez une phrase par terme.
2. Le mot de trop. « DuckDB est un SQL. » Corrigez la phrase et dites quelle confusion elle trahit.
3. Le tableur. Donnez trois choses qu'un RDBMS garantit et qu'un tableur ne garantit pas.
4. Le R de RDBMS. À quoi renvoie-t-il exactement, et par quel mécanisme concret cette propriété se manifeste-t-elle dans une base ?
5. Les cinq familles. Nommez-les par leur sigle et donnez une commande représentative de chacune.
6. L'annulation. Le DML s'annule tant qu'on n'a pas validé. Qu'en est-il du DDL sur les deux moteurs de cette formation, et ailleurs ? Quelle conséquence pratique en tirez-vous avant de taper DROP ?

7. Le classement de `SELECT`. Certains comptent quatre familles au lieu de cinq. Quelle commande change de camp, et quelle version vaut-il mieux donner en entretien ?
 8. `TRUNCATE` et `DELETE`. Les deux vident une table. Dans quelle famille se range chacun, et qu'est-ce que cette différence de famille change en pratique ? Dites aussi ce qu'elle ne change pas sur nos deux moteurs.
 9. La norme. SQL est normalisé, et pourtant les moteurs diffèrent. Expliquez ce paradoxe apparent en une phrase, puis donnez un exemple vu dans ce cours.
 10. OLTP et OLAP. Donnez la vocation de chacun en une phrase, et le nom du moteur de cette formation qui représente chaque camp. Situez ensuite les entrepôts de données dans le cloud par rapport à ces deux-là.
 11. La clé primaire. Que garantit-elle, et qu'est-ce qu'une clé de substitution dans un entrepôt ?
 12. La clé étrangère. Que garantit-elle, et quelle opération du langage cette garantie rend-elle possible ?
 13. Le schéma. Dans `dbo.DimProduct`, que désigne `dbo`, et pourquoi ce préfixe existe-t-il ?
 14. L'ordre des lignes. Une table a-t-elle un ordre ? Que se passe-t-il si vous omettez `ORDER BY` et que vous vous fiez à ce que vous voyez ?
 15. L'étoile. Décrivez la forme d'un schéma en étoile en nommant ses deux sortes de tables, et dites ce que chacune contient.
 16. L'ordre d'exécution. Donnez les sept étapes dans l'ordre où le moteur les exécute.
 17. Le nom inconnu. Vous créez un alias dans le `SELECT` et vous l'utilisez dans le `WHERE`. PostgreSQL refuse, DuckDB accepte. Expliquez qui suit la norme et pourquoi, puis dites pourquoi le même alias fonctionne dans le `ORDER BY` des deux côtés.
1. `WHERE` ou `HAVING`. Donnez la règle de partage en une phrase, en la rattachant à l'ordre d'exécution plutôt qu'à une convention.
 2. Les trois jointures. Nommez-les et dites ce que chacune garde.
 3. Le piège du `LEFT JOIN`. Décrivez-le, puis donnez la règle qui dit où placer une condition, dans le `ON` ou dans le `WHERE`.
 4. `NULL`. Qu'est-ce que ce n'est pas, et que vaut une comparaison avec `NULL` ?
 5. Les trois pièges de `NULL`. Nommez-les et donnez la parade de chacun.
 6. Les totaux et les `NULL`. Que valent `COUNT(*)` et `COUNT(Colors)` sur une table de 500 produits dont 250 ont une couleur à `NULL` ? Et `AVG(ListPrice)` si dix produits ont un prix à `NULL` ?
 7. Le monde d'à côté. Sur la version transactionnelle d'AdventureWorks, les ventes ne vivent pas dans une table de faits. Où vivent-elles, et qu'est-ce que cela change au nombre de jointures d'une question analytique ?

8. NULL ou « NA ». Dans DimProduct, la colonne Color ne contient aucun NULL, mais vaut « NA » sur 254 lignes. Qu'est-ce qu'une valeur sentinelle, pourquoi un entrepôt en met-il dans ses dimensions, et que perd-on à le faire ?

Questions pratiques

Écrivez la requête complète avant de regarder. Tapez-la vraiment dans la ligne de commande DuckDB, sur l'entrepôt ouvert par `USE dbo` ; le SQL s'apprend avec les doigts.

- P1. [OLAP] Les produits noirs. Affichez le nom et le prix des produits de couleur 'Black', du plus cher au moins cher.
- P2. [OLAP] Le nombre de produits par sous-catégorie. Pour chaque sous-catégorie, affichez son nom et le nombre de produits qu'elle contient.
- P3. [OLAP] Les produits sans sous-catégorie. Listez les noms des produits dont la sous-catégorie n'est pas renseignée.
- P4. [OLAP] Compter les vraies couleurs. En une seule requête, affichez le nombre total de lignes de DimProduct et le nombre de lignes qui ont vraiment une couleur.
- P5. [OLAP] Le filtre et le total. Affichez les produits dont le prix dépasse 1 000 dollars, avec leur sous-catégorie en toutes lettres. Les produits sans sous-catégorie doivent apparaître quand même, avec une mention lisible plutôt qu'un NULL.
- P6. [OLAP] La question analytique type. Affichez les dix produits les plus vendus en ligne, par quantité totale, avec leur nom lisible. Dites ensuite quelles tables vous avez dû joindre et pourquoi.
- P7. [OLAP] Le filtre au bon étage. Affichez les sous-catégories qui comptent plus de quinze produits plus chers que la moyenne du catalogue, avec ce compte. Attention : il y a deux filtres, et ils ne vont pas au même endroit. Dites ensuite, en une phrase, ce que le premier filtre fait au compte du second.
- P8. [OLAP] La preuve du piège. Écrivez deux requêtes qui comptent les lignes sans prix de liste, l'une avec `IS NULL`, l'autre avec `= NULL`. Exécutez les deux et notez les deux résultats. Refaites ensuite l'exercice sur Color, et expliquez pourquoi les deux zéros obtenus ne veulent pas dire la même chose.
- P9. [OLAP] Le LEFT JOIN dégradé. Écrivez une requête qui liste tous les produits avec leur sous-catégorie, puis ajoutez une condition sur la sous-catégorie dans le WHERE. Comptez les lignes avant et après, expliquez l'écart, puis corrigez pour garder tous les produits.
- P10. (avancé) La division protégée. Pour chaque produit, affichez le rapport entre son prix de liste et son coût standard (StandardCost). Rien ne garantit qu'un coût ne vaudra jamais zéro : traitez le cas proprement, dites ce qui se passerait sans cette précaution, puis vérifiez combien de coûts valent zéro et combien sont NULL dans cette base.
- P11. (avancé) Le compte qui ment. Écrivez une requête qui joint DimProduct à FactInternetSales et qui compte les produits. Comparez le résultat à un simple `COUNT(*) FROM dbo.DimProduct`. Expliquez l'écart dans les deux sens, c'est-à-dire pourquoi le compte joint peut être à la fois plus grand et plus petit.
- P12. (avancé) Trois façons de compter la même chose. Sur DimProduct, écrivez trois requêtes qui donnent le nombre de lignes ayant une vraie couleur, « NA » exclu, en utilisant trois mécanismes différents. Vérifiez qu'elles concordent, puis dites laquelle vous préférez et pourquoi.

Diagnostics

On vous montre du code qui ne fait pas ce que son auteur croit. Dites ce qui cloche, pourquoi, et donnez le correctif.

E1. Un débutant veut les produits dont le prix dépasse le double du coût :

```
SELECT EnglishProductName, ListPrice, StandardCost,  
       ListPrice / StandardCost AS ratio  
FROM   dbo.DimProduct  
WHERE  ratio > 2;
```

Sur DuckDB, elle passe. Recopiée dans PostgreSQL sur une table de même forme, elle échoue : le moteur répond que la colonne `ratio` n'existe pas. Le débutant conclut que PostgreSQL est capricieux. Expliquez la vraie cause, dites pourquoi le message est trompeur, donnez les deux corrections possibles, et signalez un second défaut de cette requête qui n'a rien à voir avec le premier. Ce second défaut ne lève aucune erreur sur l'entrepôt : dites ce qu'il y produit à la place.

E2. Une analyste veut les sous-catégories qui vendent pour plus de 100 000 dollars :

```
SELECT s.EnglishProductSubcategoryName, SUM(f.SalesAmount) AS ventes  
FROM   dbo.FactInternetSales AS f  
JOIN   dbo.DimProduct          AS p ON p.ProductKey = f.ProductKey  
JOIN   dbo.DimProductSubcategory AS s ON s.ProductSubcategoryKey = p.ProductSubcategoryKey  
WHERE  SUM(f.SalesAmount) > 100000  
GROUP BY s.EnglishProductSubcategoryName;
```

Le moteur refuse. Expliquez pourquoi en vous appuyant sur l'ordre d'exécution, corrigez, puis dites ce qui changerait si elle voulait au contraire ne compter que les ventes postérieures à 2012.

E3. Un développeur veut savoir si des sous-catégories ne sont rattachées à aucun produit :

```
SELECT EnglishProductSubcategoryName  
FROM   dbo.DimProductSubcategory  
WHERE  ProductSubcategoryKey NOT IN (SELECT ProductSubcategoryKey FROM dbo.DimProduct);
```

Le résultat est vide, et il en conclut que toutes les sous-catégories sont utilisées. A-t-il le droit de conclure ça ? Expliquez le mécanisme exact, proposez une manipulation en deux lignes qui prouve que la requête est incapable de répondre quoi qu'il arrive, puis donnez deux écritures correctes.

E4. Une analyste veut savoir quelle part du catalogue n'a pas de couleur renseignée :

```
SELECT COUNT(*) FROM dbo.DimProduct WHERE Color IS NULL;
```

Elle obtient zéro, et écrit dans son rapport que toutes les couleurs sont renseignées. Dites ce qui cloche, par quelle requête elle aurait pu s'en apercevoir, et corrigez. Dites ensuite pourquoi le chiffre corrigé mérite lui-même une nuance avant d'entrer dans le rapport.

Questions à développer

Un court paragraphe de raisonnement, pas un mot juste.

A. L'ordre d'exécution comme outil de diagnostic. On présente souvent l'ordre d'exécution comme une liste à mémoriser. Défendez l'idée que c'est en réalité un outil de raisonnement, et montrez-le : prenez trois erreurs de débutant qui semblent sans rapport entre elles, et expliquez comment la même question, « à cette étape, cette colonne existe-t-elle », les résout toutes les trois. Terminez en disant ce que cette grille ne permet pas d'expliquer.

B. Pourquoi deux modèles pour la même entreprise. AdventureWorks existe en version transactionnelle normalisée et en version entrepôt dénormalisée. Expliquez à quelqu'un qui trouve ça redondant pourquoi les deux existent, ce que chacune optimise, et ce qu'elle sacrifie pour y arriver. Prenez un exemple concret d'opération qui est facile d'un côté et pénible de l'autre, dans les deux sens.

C. NULL, une bonne ou une mauvaise idée. La valeur NULL cause une part notable des bogues de SQL, au point que certains auteurs recommandent de l'interdire dans les modèles. Présentez honnêtement les deux positions : ce que NULL apporte que rien d'autre n'apporte, et ce qu'il coûte en pièges. Concluez par la règle que vous adopteriez en concevant une table.

Mises en situation

On décrit une entreprise et une question métier. À vous de choisir l'approche, de l'écrire, et surtout de la justifier. Il y a plusieurs bonnes réponses, mais il y a des réponses indéfendables.

M1. Librairie Rive-Gauche, l'inventaire qui ne balance pas. La librairie tient son stock dans une petite base PostgreSQL. Chaque soir, un employé lance une requête qui compte les livres en stock par rayon, et le total ne correspond jamais tout à fait à l'inventaire physique. En regardant la table, vous voyez que la colonne rayon est parfois vide, que certains livres ont une quantité à NULL plutôt qu'à zéro, et que la requête utilise un JOIN vers la table des rayons.

Expliquez, sans écrire une ligne de SQL pour l'instant, les trois mécanismes distincts par lesquels ce rapport peut perdre des livres. Puis écrivez la requête corrigée, et dites pour chaque correction lequel des trois mécanismes elle règle. Terminez par la question de fond : faut-il corriger la requête, ou faut-il corriger les données, et qu'est-ce qui décide ?

M2. Commission scolaire des Deux-Rives, la question de vocabulaire. Vous arrivez dans une équipe qui utilise indifféremment les mots « base de données », « serveur », « SQL » et « Oracle » pour désigner la même chose. On vous demande de préparer une page pour les nouveaux arrivants.

Rédigez le raisonnement que vous suivriez : quelles distinctions sont indispensables, lesquelles sont du pinaillage, et comment vous vous y prenez pour qu'une clarification de vocabulaire ne passe pas pour une leçon. Justifiez notamment pourquoi la confusion entre le langage et le moteur a des conséquences concrètes, avec un exemple où elle mène à une mauvaise décision technique.

M3. (avancé) Distribution alimentaire Beauharnois, la mauvaise table. L'entreprise a un entrepôt de données alimenté chaque nuit et une base transactionnelle de production. Un analyste vient de brancher son tableau de bord directement sur la base de production, parce que, dit-il, « les données sont plus fraîches et le résultat est le même ». Son tableau tourne toutes les cinq minutes et interroge trois ans d'historique de commandes.

Répondez-lui. Vous devez traiter quatre choses : ce qui est juste dans son raisonnement, ce qui est faux, ce que sa décision coûte à des gens qui ne sont pas dans la conversation, et à quelles conditions précises il aurait raison. Terminez en formulant la question qu'il aurait dû se poser en premier, et dites pourquoi c'est celle-là et pas la question de la fraîcheur.